

Create a comprehensive and professional `README.md` for the following semester PBL project.

Project Title

Git for Embeddings: Version Control and Incremental Management System for Vector Embeddings

Alternative possible names:

- EmbedGit
- VectorVersion
- EmbeddingVC
- Vectory
- Git-Like Version Control for Embeddings

The README should present this as both:

1. A practical AI and cloud engineering project.
2. A research-oriented system that investigates how embedding datasets can be versioned, compared, updated, restored, and reproduced efficiently.

Background

Modern AI applications such as semantic search engines, recommendation systems, Retrieval-Augmented Generation systems, document retrieval systems, and vector databases depend on embeddings.

An embedding converts an object such as text, an image, audio, or a document into a numerical vector.

For example:

```
Document
  |
  v
Embedding model
  |
  v
[0.12, -0.43, 0.87, ...]
```

These vectors are stored inside vector databases such as:

- ChromaDB
- FAISS
- Pinecone

- Weaviate
- Milvus
- Qdrant

However, embeddings are not static.

They may change when:

- the original document changes
- preprocessing logic changes
- chunk size changes
- chunk overlap changes
- the embedding model changes
- the embedding model version changes
- metadata changes
- documents are added or deleted
- incorrect embeddings need to be restored
- the vector database is rebuilt

Unlike source code, embeddings usually do not have a general-purpose version-control system comparable to Git.

Teams often regenerate complete embedding collections, overwrite existing vectors, create separate database collections, or rely on custom internal pipelines.

This project proposes a Git-like system for managing embedding datasets.

Core Problem

Regenerating an entire embedding collection can be expensive and slow.

Consider a large RAG system containing hundreds of thousands or millions of document chunks.

When only a few documents change, recreating all embeddings is wasteful.

The system should identify:

- which documents were added
- which documents were modified
- which documents were deleted
- which chunks were affected
- which embeddings need regeneration
- which embeddings can be reused safely

The system should then update only the affected embeddings.

The project must clearly distinguish two different cases.

Case 1: Source data or preprocessing changes

If only documents, metadata, chunking, or preprocessing change while the embedding model remains the same, unchanged embeddings can generally be reused.

Incremental regeneration is practical in this case.

Case 2: Embedding model changes

If the embedding model changes completely, existing vectors may no longer be directly compatible with the new vector space.

For example:

Model A embeddings:
384 dimensions

Model B embeddings:
768 dimensions

Even when two models have the same vector dimension, their vector spaces may represent semantic relationships differently.

Therefore, the system should not blindly mix embeddings from different models.

The first version of the project should treat a model change as a major version change that normally requires regeneration.

Advanced research can later investigate whether old vector spaces can be mapped into new vector spaces.

Problem Statement

Embedding-based systems lack a standardized and general-purpose mechanism for:

- tracking changes in source documents
- versioning embedding collections
- identifying stale vectors
- incrementally regenerating affected embeddings
- comparing embedding versions
- restoring previous versions
- reproducing an embedding dataset

- maintaining metadata about models and preprocessing
- measuring semantic drift between versions

The project aims to build a lightweight, Git-inspired version-control system for embeddings that avoids unnecessary regeneration and improves reproducibility.

Main Research Question

Can a Git-like version-control system reduce the computational cost and time required to update large embedding collections while preserving retrieval quality and reproducibility?

Secondary Research Questions

The project should investigate questions such as:

1. How much time can incremental regeneration save compared with full regeneration?
 2. How much storage overhead is introduced by maintaining multiple embedding versions?
 3. How can the system reliably detect which embeddings are stale?
 4. What information is required to reproduce an embedding version?
 5. How should the system behave when the embedding model changes?
 6. Can semantic drift between two embedding versions be measured?
 7. Can similar embedding spaces be aligned to reduce complete regeneration?
 8. When is reuse of an old embedding safe, and when is regeneration mandatory?
 9. How do chunking and preprocessing changes affect embedding reuse?
 10. How does versioning affect downstream retrieval accuracy?
-

Project Objectives

The project should aim to:

- create repositories for embedding-based projects
- track source files and document chunks
- calculate hashes for documents and chunks
- detect added, modified, deleted, and unchanged content
- generate embeddings only for affected chunks
- store model and preprocessing configuration
- create immutable embedding commits
- support branches or experiment versions
- compare two embedding versions
- restore an earlier embedding state
- integrate with a local vector database

- measure regeneration time and computational savings
 - evaluate retrieval quality after incremental updates
 - provide a command-line interface
 - optionally provide a dashboard for visualizing changes
-

Git-Inspired Concepts

The system should adapt selected Git concepts to embedding workflows.

Repository

An embedding repository contains:

- source documents
- chunk information
- embeddings
- metadata
- model configuration
- preprocessing configuration
- commit history

Example:

```
embedding-project/  
|  
|-- documents/  
|-- .embedgit/  
|   |-- config.json  
|   |-- index.json  
|   |-- objects/  
|   |-- commits/  
|   |-- refs/  
|   |-- cache/  
|  
|-- vector_store/  
|-- embedgit.yaml
```

Init

Initializes an embedding repository.

Example:

```
embedgit init
```

Add

Tracks new or modified documents.

```
embedgit add documents/
```

Status

Displays changes since the most recent commit.

Example:

```
Modified documents:
  notes/chapter_3.pdf

Added documents:
  notes/chapter_4.pdf

Deleted documents:
  notes/old_appendix.pdf

Unchanged chunks:
  1,240

Chunks requiring regeneration:
  86
```

Command:

```
embedgit status
```

Commit

Creates an immutable snapshot containing:

- document hashes
- chunk hashes
- embedding references
- embedding model identity

- model version
- vector dimension
- preprocessing configuration
- chunking configuration
- metadata schema
- timestamp
- parent commit
- optional commit message

Example:

```
embedgit commit -m "Added chapter four notes"
```

Log

Displays embedding version history.

```
embedgit log
```

Diff

Compares two embedding commits.

It should report changes such as:

- documents added
- documents removed
- documents modified
- chunks added
- chunks removed
- chunks regenerated
- embeddings reused
- model changes
- preprocessing changes
- vector dimensionality changes
- semantic similarity changes
- storage difference

Example:

```
embedgit diff commit_a commit_b
```

Checkout or Restore

Restores a previous embedding version.

```
embedgit checkout <commit-id>
```

or:

```
embedgit restore <commit-id>
```

Branch

Supports experiments with different:

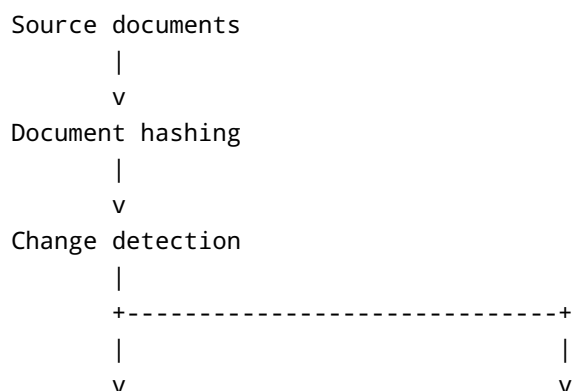
- embedding models
- chunk sizes
- preprocessing methods
- metadata configurations
- retrieval strategies

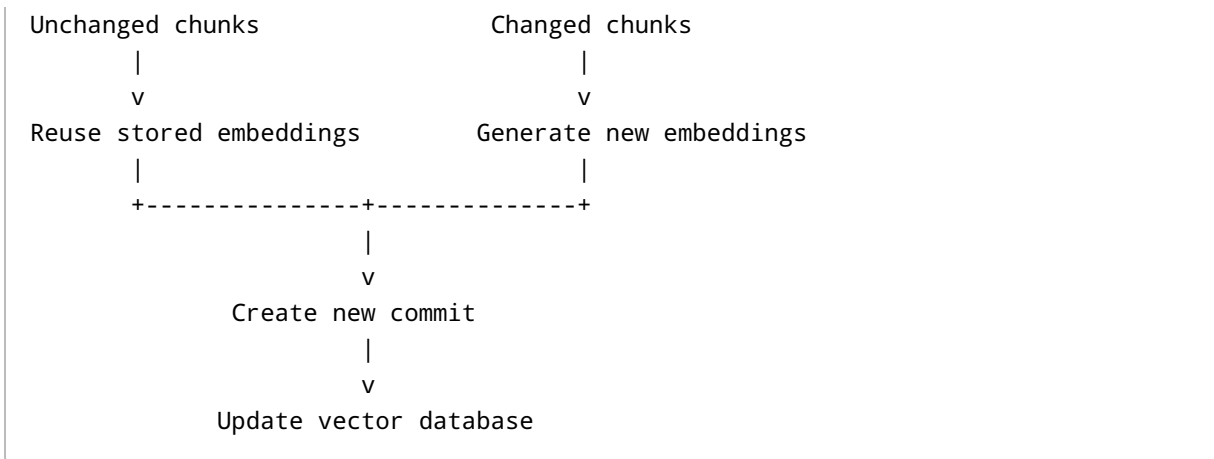
Example:

```
embedgit branch bge-experiment  
embedgit checkout bge-experiment
```

Proposed Workflow

The README should include a workflow similar to the following:





Change Detection Strategy

The system should not compare only filenames.

It should use content-based hashes.

Possible levels of hashing:

Document-level hashing

Hash the entire source document.

Useful for quickly identifying whether a document changed.

```
document_hash = SHA256(document_content)
```

Chunk-level hashing

Hash each normalized chunk.

```
chunk_hash = SHA256(
    normalized_chunk_text
    + preprocessing_config
    + chunking_config
)
```

Chunk-level hashing allows reuse when only a small part of a document changes.

For example, if one paragraph changes, unaffected chunks can retain their existing embeddings.

Embedding configuration hashing

Create a fingerprint from:

- embedding model name
- exact model version or revision
- tokenizer version
- vector dimension
- normalization settings
- pooling method
- preprocessing pipeline
- chunk size
- chunk overlap

Example:

```
embedding_config_hash = SHA256(  
    model_name  
    + model_revision  
    + tokenizer_version  
    + preprocessing_config  
    + chunking_config  
)
```

An embedding may be reused only when both conditions are satisfied:

```
chunk_hash is unchanged  
AND  
embedding_config_hash is unchanged
```

Content-Addressable Storage

The project should use content-addressable storage inspired by Git.

Instead of duplicating embeddings for every commit, embeddings should be stored using a hash-based identifier.

Example:

```
.embedgit/objects/  
  ab/  
    cdef123456...
```

Each embedding object may contain:

```
{  
  "chunk_hash": "abc123",  
  "model": "BAAI/bge-small-en-v1.5",  
  "model_revision": "specific-revision",  
  "dimension": 384,  
  "normalized": true,  
  "embedding": [0.12, -0.43, 0.87],  
  "created_at": "timestamp"  
}
```

Multiple commits can reference the same embedding object.

This prevents unchanged embeddings from being duplicated.

The README should explain that the project is not expected to calculate Git-style textual deltas between floating-point vectors initially.

The first version should focus on:

- deduplication
- immutable objects
- shared references
- incremental regeneration
- metadata tracking

Vector delta compression can be treated as a future research enhancement.

Proposed Architecture

Include a Mermaid architecture diagram similar to:

```
flowchart TD  
  A[Source Documents] --> B[Document Loader]  
  B --> C[Preprocessing and Chunking]  
  C --> D[Chunk Hash Generator]  
  D --> E[Change Detection Engine]
```

```
E -->|Unchanged Chunk| F[Embedding Object Cache]
E -->|New or Modified Chunk| G[Embedding Model]

G --> H[New Embedding Object]
F --> I[Commit Builder]
H --> I

I --> J[Commit Manifest]
J --> K[Vector Store Synchronizer]
K --> L[(ChromaDB / FAISS / Qdrant)]

J --> M[Version History]
M --> N[Diff and Restore Engine]
```

Core Components

1. Repository Manager

Responsible for:

- initialization
- repository configuration
- commit history
- branches
- references
- repository validation

2. Document Tracker

Responsible for:

- scanning source directories
- detecting file additions
- detecting modifications
- detecting deletions
- creating document hashes
- maintaining document metadata

3. Chunking Engine

Responsible for:

- text extraction

- normalization
- chunk creation
- chunk IDs
- chunk hashes
- source-to-chunk mappings

4. Embedding Engine

Responsible for:

- loading the embedding model
- embedding new chunks
- batching requests
- normalizing vectors
- storing model metadata
- avoiding redundant computation

5. Change Detection Engine

Compares the working state with the latest commit.

Classifies chunks as:

ADDED
MODIFIED
DELETED
UNCHANGED
STALE

A chunk should be marked as **STALE** when its text is unchanged but the embedding configuration has changed.

6. Object Store

Stores immutable:

- embedding objects
- chunk objects
- manifests
- configuration objects

7. Commit Manager

Creates version snapshots.

Each commit references objects rather than duplicating all data.

8. Vector Store Adapter

Synchronizes commits with a vector database.

The first implementation may support ChromaDB.

The architecture should allow adapters for:

- FAISS
- Qdrant
- Pinecone
- Milvus
- Weaviate

9. Diff Engine

Compares versions at multiple levels.

Structural diff

- documents added or removed
- chunks added or removed
- configuration changes

Embedding diff

- cosine similarity between old and new embeddings
- Euclidean distance
- nearest-neighbour consistency
- number of significantly changed vectors

Retrieval diff

- changes in retrieved top-k results
- ranking changes
- Recall@K changes
- NDCG changes
- Mean Reciprocal Rank changes

10. Restore Engine

Restores:

- commit manifests
- embedding object references

- vector-store state
 - configuration state
-

Minimum Viable Product

The MVP should remain achievable within one semester.

The MVP should include:

1. Local repository initialization.
2. Text, Markdown, and PDF document ingestion.
3. Text extraction and chunking.
4. SHA-256 document and chunk hashes.
5. Embedding generation using one sentence-transformer model.
6. Persistent embedding storage.
7. ChromaDB integration.
8. Detection of added, modified, deleted, unchanged, and stale chunks.
9. Incremental regeneration.
10. Commit creation.
11. Commit history.
12. Diff between two commits.
13. Restore or checkout of an earlier version.
14. CLI commands.
15. Experimental comparison between full and incremental regeneration.

The MVP does not need:

- distributed storage
 - collaboration between multiple users
 - automatic vector-space alignment
 - GPU cluster support
 - complex vector delta compression
 - a production-ready hosted service
-

Example CLI

The README should provide a possible command interface.

```
# Create a repository
embedgit init

# Configure the embedding model
```

```
embedgit config set model BAAI/bge-small-en-v1.5

# Configure chunking
embedgit config set chunk_size 500
embedgit config set chunk_overlap 50

# Add documents
embedgit add ./documents

# View detected changes
embedgit status

# Generate only required embeddings
embedgit embed

# Create a snapshot
embedgit commit -m "Initial knowledge-base embeddings"

# View history
embedgit log

# Compare versions
embedgit diff HEAD~1 HEAD

# Restore an older version
embedgit checkout <commit-id>

# Create an experimental branch
embedgit branch minilm-experiment

# Benchmark incremental regeneration
embedgit benchmark
```

Example Repository Metadata

```
repository:
  name: student-notes-rag
  version: 1

embedding:
  provider: sentence-transformers
  model: BAAI/bge-small-en-v1.5
  revision: pinned-model-revision
  dimension: 384
```



```
    normalize_embeddings: true

chunking:
  strategy: recursive-character
  chunk_size: 500
  chunk_overlap: 50

preprocessing:
  lowercase: false
  remove_extra_whitespace: true
  unicode_normalization: NFKC

vector_store:
  provider: chromadb
  collection: student-notes
  persist_directory: ./vector_store
```

Commit Manifest Example

```
{
  "commit_id": "91b7e92",
  "parent_commit": "73ac120",
  "message": "Updated operating systems notes",
  "timestamp": "2026-08-04T12:00:00Z",
  "embedding_config_hash": "config123",
  "documents": {
    "os/chapter1.pdf": {
      "document_hash": "doc123",
      "chunks": [
        {
          "chunk_id": "chunk-001",
          "chunk_hash": "chunkhash001",
          "embedding_object": "embedobj001"
        }
      ]
    }
  },
  "statistics": {
    "documents_added": 0,
    "documents_modified": 1,
    "documents_deleted": 0,
    "chunks_reused": 92,
    "chunks_regenerated": 8
  }
}
```

```
}  
}
```

Experimental Evaluation

The project should include a proper experiment rather than only demonstrating CLI commands.

Create a document collection containing many documents and chunks.

Run controlled modifications such as:

- modifying 1% of documents
- modifying 5% of documents
- modifying 10% of documents
- adding new documents
- deleting documents
- changing metadata only
- changing chunk size
- changing chunk overlap
- changing the embedding model

Compare:

Baseline 1: Full regeneration

Every document is processed and embedded again.

Baseline 2: Document-level incremental regeneration

Only modified documents are re-embedded completely.

Proposed method: Chunk-level incremental regeneration

Only new or changed chunks are embedded.

Measure:

- total execution time
- embedding generation time
- number of embedding-model calls
- number of vectors regenerated
- number of vectors reused
- CPU usage

- memory usage
- storage overhead
- estimated API cost, if using a paid embedding API
- vector-store update time
- retrieval quality
- reproducibility

Evaluation Metrics

Efficiency metrics

Regeneration Reduction =
 $1 - \text{regenerated_chunks} / \text{total_chunks}$

Time Saved =
 $\text{full_regeneration_time} - \text{incremental_regeneration_time}$

Speedup =
 $\text{full_regeneration_time} / \text{incremental_regeneration_time}$

Embedding Reuse Rate =
 $\text{reused_embeddings} / \text{total_embeddings_in_new_version}$

Storage Overhead =
 $\text{versioned_storage_size} - \text{single_version_storage_size}$

Retrieval metrics

Use a small set of queries with relevant-document labels.

Measure:

- Recall@K
- Precision@K
- Mean Reciprocal Rank
- NDCG@K
- top-k overlap
- ranking stability

The purpose is to verify that incremental updates produce the same or nearly the same retrieval results as complete regeneration when configurations are unchanged.

Reproducibility metrics

Verify whether the same commit can recreate:

- the same chunks
- the same metadata
- the same model configuration
- the same vector-store contents
- sufficiently similar embeddings

Exact floating-point equality may not always be guaranteed across different hardware or library versions.

The README should acknowledge this limitation.

Important Research Boundary

The README must clearly state that this project does not assume that embeddings from completely different models can be freely reused.

Changing the embedding model may produce a different vector space.

For the MVP:

```
Same chunk + same embedding configuration
  -> reuse embedding

Changed chunk + same embedding configuration
  -> regenerate affected embedding

Same chunk + changed embedding configuration
  -> mark stale and regenerate

Changed embedding model
  -> create a major version or experimental branch
```

This rule is important for system correctness.

Advanced Research Direction: Vector-Space Alignment

An optional later research direction is to investigate whether embeddings produced by an old model can be mapped into the vector space of a new model.

Suppose a set of shared anchor documents is embedded using both models:

Old model:
 X_{old}

New model:
 X_{new}

The project may attempt to learn a transformation:

$$X_{new} \approx f(X_{old})$$

Possible methods include:

- linear regression
- ridge regression
- orthogonal Procrustes alignment
- canonical correlation analysis
- small neural networks
- knowledge distillation
- contrastive alignment
- transfer learning

However, this should be presented as an experimental research extension rather than a guaranteed solution.

The system must evaluate whether transformed vectors preserve:

- nearest neighbours
- semantic similarity
- retrieval ranking
- downstream RAG quality

This approach may fail when the two models have substantially different semantic structures.

Even if vector alignment is explored, complete regeneration remains the correctness baseline.

Other Possible Research Extensions

Semantic drift detection

Measure how much embeddings change across versions.

For each shared chunk:

```
drift = 1 - cosine_similarity(old_embedding, new_embedding)
```

Large drift may indicate:

- a model change
- preprocessing changes
- document meaning changes
- unstable embeddings

Retrieval-aware version comparison

Instead of only comparing vectors, compare how search results change between versions.

Storage optimization

Explore:

- vector quantization
- float16 storage
- int8 quantization
- product quantization
- compressed snapshots
- delta encoding
- deduplicated object storage

Branch merging

Investigate how to merge embedding branches.

Possible conflicts:

- same document changed differently
- incompatible embedding models
- different chunking configurations
- metadata-schema differences

Remote repositories

Allow embedding repositories to be pushed to cloud storage such as:

- AWS S3
- Google Cloud Storage
- Azure Blob Storage
- MinIO

Collaborative workflows

Support multiple researchers or developers working on different embedding experiments.

Experiment tracking integration

Integrate with tools such as:

- MLflow
- Weights & Biases
- DVC

Clarify that these tools may track datasets and experiments, while this project focuses specifically on embedding objects, vector-store synchronization, and embedding-aware diffs.

Suggested Tech Stack

Component	Technology
Programming language	Python
CLI	Typer or Click
Embedding models	Sentence Transformers
Initial model	BAAI/bge-small-en-v1.5 or all-MiniLM-L6-v2
Vector database	ChromaDB
Hashing	SHA-256
Metadata	JSON or SQLite
Object storage	Local filesystem
Document loading	PyMuPDF, Markdown, plain text
Validation	Pydantic

Component	Technology
Testing	Pytest
Dashboard	Streamlit, optional
Containerization	Docker
Cloud storage extension	AWS S3 or MinIO
Experiment tracking	MLflow, optional

Possible Project Structure

```

embedgit/
|
|-- README.md
|-- pyproject.toml
|-- requirements.txt
|-- Dockerfile
|
|-- src/
|   |-- embedgit/
|       |-- __init__.py
|       |-- cli.py
|       |-- config.py
|       |-- repository.py
|       |-- hashing.py
|       |-- document_loader.py
|       |-- chunking.py
|       |-- embedding_engine.py
|       |-- object_store.py
|       |-- change_detector.py
|       |-- commit_manager.py
|       |-- diff_engine.py
|       |-- restore_engine.py
|       |-- vector_store/
|           |-- base.py
|           |-- chromadb_adapter.py
|       |-- models/
|           |-- commit.py
|           |-- document.py
|           |-- embedding.py
|
|-- tests/
|   |-- test_hashing.py

```



```
| |-- test_change_detection.py
| |-- test_commits.py
| |-- test_restore.py
|
|-- examples/
| |-- student_notes_rag/
|
|-- benchmarks/
| |-- generate_dataset.py
| |-- run_full_regeneration.py
| |-- run_incremental_regeneration.py
| |-- evaluate_retrieval.py
|
|-- docs/
| |-- architecture.md
| |-- research_methodology.md
| |-- results.md
```

Semester Development Plan

Phase 1: Research and system design

- review existing vector databases
- study Git object storage
- study dataset-versioning tools
- define repository schema
- define commit structure
- define reuse and invalidation rules

Phase 2: Basic embedding pipeline

- document ingestion
- text extraction
- chunking
- embedding generation
- ChromaDB persistence

Phase 3: Version-control core

- content hashing
- status detection
- object storage
- commits
- history

- incremental embedding generation

Phase 4: Diff and restore

- compare commits
- display changed chunks
- calculate embedding drift
- restore previous versions
- synchronize ChromaDB

Phase 5: Experiments

- generate document-change scenarios
- run full regeneration
- run document-level regeneration
- run chunk-level regeneration
- measure efficiency and retrieval quality

Phase 6: Interface and documentation

- improve CLI
- add optional Streamlit dashboard
- create architecture diagrams
- create final report
- prepare demonstration

Phase 7: Optional research extension

- experiment with vector-space alignment
- evaluate transformed embeddings against full regeneration

Expected Outcome

The expected result is a working tool that:

- tracks versions of embedding datasets
- detects changed content
- reuses unchanged embeddings
- regenerates only affected vectors
- records model and preprocessing configurations
- compares embedding versions
- restores previous versions
- synchronizes with a vector database
- demonstrates lower regeneration time and computational cost than full regeneration

The expected research finding is that chunk-level incremental regeneration will provide substantial savings when only a small percentage of the source collection changes.

However, when the embedding model or preprocessing configuration changes, much or all of the embedding collection may need regeneration.

Research Contribution

The README should avoid claiming that the project invents dataset version control or vector databases.

Its proposed contribution is:

A lightweight Git-inspired version-control abstraction specifically designed for embedding collections, combining content-addressable storage, chunk-level change detection, embedding reuse, configuration-aware invalidation, embedding-aware diffs, vector-store synchronization, and reproducible snapshots.

The main contribution is not a new embedding model.

It is a system for managing the lifecycle of embeddings.

Scope and Feasibility

The README should explain that the full research problem is large.

A production-grade version would require:

- distributed object storage
- concurrency control
- remote collaboration
- authentication
- large-scale database integrations
- fault tolerance
- cross-platform reproducibility
- model registry integration

The semester project should focus on a local single-user implementation using one embedding model and ChromaDB.

The system should be designed so that additional backends can be added later.

Limitations

Include limitations such as:

- model changes often require complete regeneration
 - exact reproducibility may depend on hardware and library versions
 - PDF extraction may produce unstable text
 - small text edits may change chunk boundaries
 - chunk-level reuse depends on deterministic chunking
 - storing multiple embedding versions introduces storage overhead
 - semantic similarity does not guarantee retrieval equivalence
 - vector-space mapping may not work across unrelated models
 - local benchmarks may not represent production-scale systems
 - external embedding APIs may produce version-dependent results
-

Example Use Case

Use a university notes RAG system as an example.

Suppose the knowledge base contains:

```
10,000 documents
100,000 chunks
100,000 embeddings
```

A student modifies 50 documents.

Without version control:

```
All 100,000 embeddings may be regenerated.
```

With document-level regeneration:

```
All chunks belonging to the 50 modified documents are regenerated.
```

With chunk-level EmbedGit tracking:

```
Only the chunks whose normalized content changed are regenerated.
```

The system creates a new commit while continuing to reference the unchanged embedding objects from the previous commit.

This saves computation, time, API cost, and storage.

README Requirements

The final README must include:

- project title
- badges or badge placeholders
- project overview
- motivation
- problem statement
- research questions
- objectives
- key terminology
- Git-to-embedding concept mapping
- architecture diagram using Mermaid
- workflow diagram using Mermaid
- core components
- versioning and invalidation rules
- CLI examples
- installation instructions
- quick-start example
- configuration example
- repository structure
- experimental methodology
- baseline strategies
- evaluation metrics
- expected results
- limitations
- future enhancements
- semester roadmap
- research contribution
- ethical and practical considerations
- contribution guidelines
- license placeholder
- project status

Use clear academic and engineering language.

Do not exaggerate novelty.

Explicitly distinguish:

- versioning source documents
- versioning embedding objects
- versioning vector-database state
- tracking embedding-model configuration

Explain that this system is inspired by Git but is not intended to replace Git.

Git can continue to manage source code, while EmbedGit manages large embedding artifacts and their relationship to documents, chunks, configurations, and vector stores.

Generate the result as a polished, complete `README.md` suitable for a university semester PBL repository.